

1. a) No because the information gain depends on the counts of labels in each split. If you duplicate a class, the counts change, thus the information gain changes, and ultimately, we are not guaranteed to have an identical decision tree.

b). Yes. Because there is no noise, we can achieve zero training error no matter what column/feature we start with. The downside of starting on a suboptimal information gain is that the tree might have to be deeper, but nonetheless it will achieve zero error in training.

c) Not in all cases because the greedy information gain splitting is calculated at that node and is not guaranteed to be the best global split. Because of this, greedily splitting on information gain does not result in the tree with the smallest training error

$$d) \quad H(Y|X=x) = - \sum_y P(Y=y|X=x) \log(P(Y=y|X=x))$$

because they're independent,
 $P(Y=y|X=x) = P(Y=y)$

$$= - \sum_y P(Y=y) \log(P(Y=y)) = H(Y)$$

$$H(Y|X) = \sum_x P(X=x) H(Y|X=x) = \sum_x \underbrace{P(X=x)}_{\text{independent}} H(Y) = H(Y)$$

$$e) \quad H(X, Y) = - \sum_{x \in X} \sum_{y \in Y} P(X=x, Y=y) \log_2 P(X=x, Y=y)$$

$$= - \sum_{x \in X} \sum_{y \in Y} P(X, Y) \log_2 (P(Y|X) P(X))$$

$$= - \sum_{x \in X} \sum_{y \in Y} P(X, Y) \log_2 P(X) - \sum_{x \in X} \sum_{y \in Y} P(X, Y) P(Y|X)$$

$$= - \sum_{x \in X} \log_2 P(X) \sum_{y \in Y} P(X, Y) - \sum_{x \in X} P(X) \sum_{y \in Y} P(Y|X) \log_2 P(Y|X)$$

$$= - \sum_{x \in X} P(X) \log_2 P(X) - \sum_{x \in X} P(X) P(Y|X=x)$$

$$= H(X) + H(Y|X)$$

$$2. a) \quad (R_n) = \left(\frac{1}{n} \sum_{i=1}^n [y^{(i)} \log(\hat{y}) + (1-y^{(i)}) \log(1-\hat{y})] \right) \frac{1}{\partial \hat{y}}$$

$$0 = -\frac{1}{n} \sum_{i=1}^n \left(\frac{y^{(i)}}{\hat{y}} - \frac{1-y^{(i)}}{1-\hat{y}} \right)$$

$$0 = \sum_{i=1}^n \frac{y^{(i)} - \hat{y}}{\hat{y}(1-\hat{y})}$$

$$0 = \sum_{i=1}^n y^{(i)} - \hat{y}$$

$$\boxed{\hat{y} = \frac{1}{n} \sum_{i=1}^n y^{(i)}}$$

$$b) \quad \hat{y} = \frac{1}{1+e^{-z}} \quad \text{and} \quad 1-\hat{y} = \frac{1}{1+e^z}$$

$$\text{loss}_{\text{BCE}}(y, \hat{y}) = -y \log(\hat{y}) - (1-y) \log(1-\hat{y})$$

$$= -y \log\left(\frac{1}{1+e^{-z}}\right) - (1-y) \log\left(\frac{1}{1+e^z}\right)$$

$$= y \log(1+e^{-z}) + (1-y) \log(1+e^z)$$

$$= \log(1+e^z) - yz$$

$$z = \log\left(\frac{\hat{y}}{1-\hat{y}}\right)$$

$$e^z = \frac{\hat{y}}{1-\hat{y}}$$

$$e^z \cdot \hat{y} = \hat{y}$$

$$e^z = \hat{y} + \hat{y}e^z = \hat{y}(1+e^z)$$

$$\frac{e^z}{1+e^z} = \hat{y}$$

$$\hat{y} = \frac{1}{1+e^{-z}} \rightarrow 1-\hat{y} = \frac{1}{1+e^z}$$

$$c) \quad \frac{d \text{loss}}{dz} = \frac{e^z}{1+e^z} - y = \hat{y} - y \quad \text{for} \quad \hat{y}_t = \frac{1}{1+e^{-z_t}}$$

$$\text{So, } \boxed{z_{t+1} = z_t - \alpha(\hat{y}_t - y)}$$

d) **upload code*

**n_estimators*. Performance on the test and train datasets increases very quickly at the start when we add more estimators and then increases logarithmically. This shows that the boosted classifier generalizes much better than a single tree when we add *estimators*.

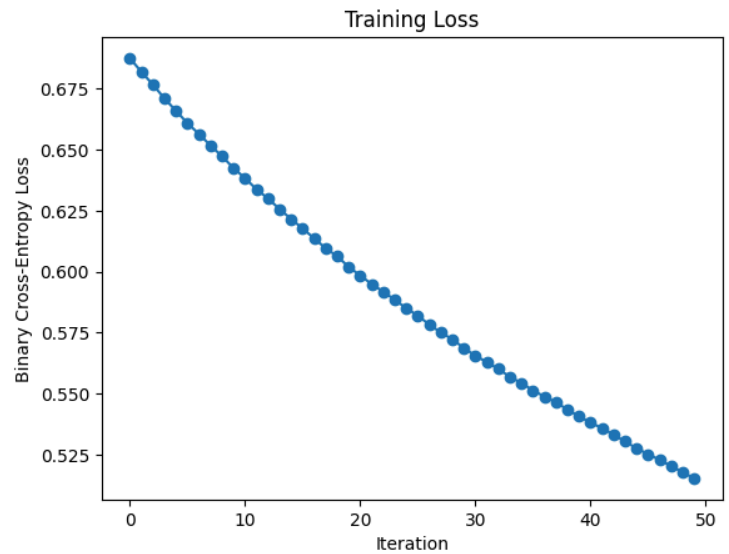
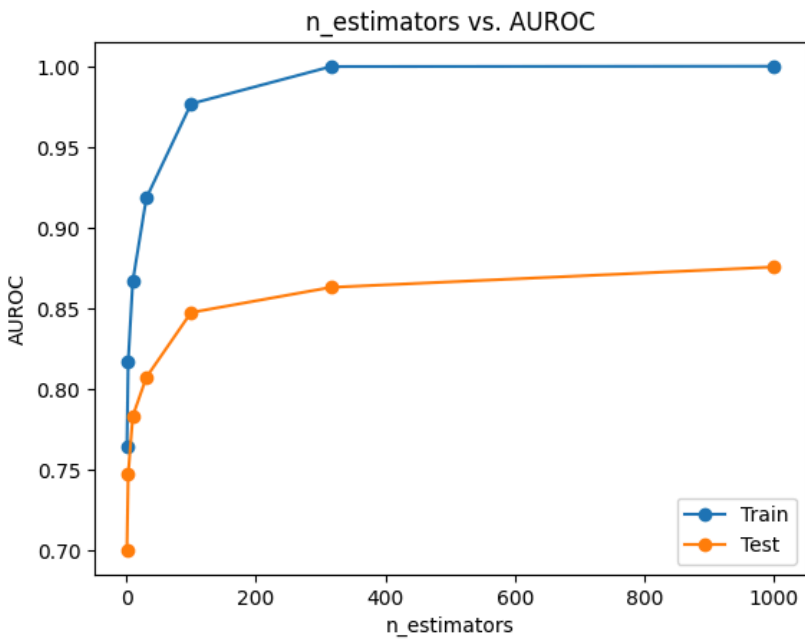
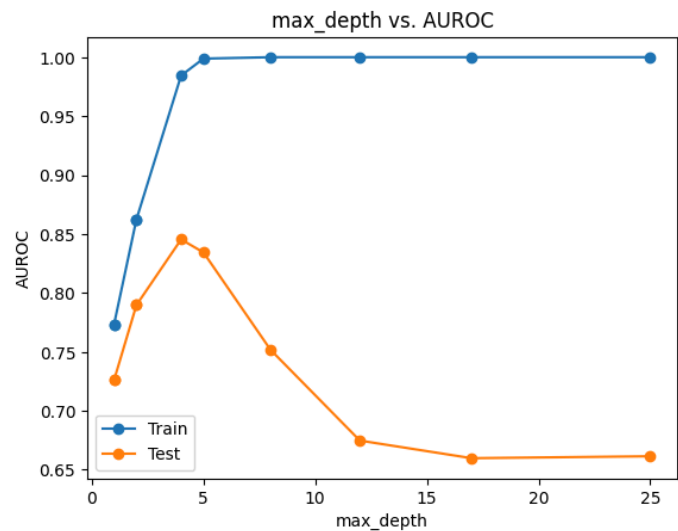
**max_depth* ii. The test performance improves with increasing tree depth to about 4 or 5 and then decreases as the model begins to overfit. *So, shallow trees generalize better and deeper trees overfit to the training data.*

**min_loss* iii. Training loss decreases almost linearly showing that each added weak learner reduces training error and the model is converging. *However, the test loss starts to increase after a certain point.*


```

26 def fit(self, X: npt.NDArray, y: npt.NDArray) -> GradientBoostingClassifier445:
27     self.init_estimator = None
28     # TODO: initialize base score to the log-odds of the positive class
29     p = np.mean(y)
30     self.init_estimator = np.log(p / (1 - p))
31
32     # initial predictions
33     F_m = np.full(y.shape, self.init_estimator)
34     self.loss = []
35     for _ in range(self.n_estimators):
36
37         # TODO: compute pseudo-residuals xx
38         # TODO: fit a DecisionTreeRegressor to the residuals with random_state=445 xx
39         # TODO: update ensemble predictions xx
40         # TODO: calculate the current loss xx
41
42         # pseudo-residuals
43         y_hat = self._sigmoid(F_m)
44         residuals = y - y_hat
45
46         # fit model
47         model = DecisionTreeRegressor(max_depth=self.max_depth, random_state=445)
48         model.fit(X, residuals)
49
50         # update preds
51         F_m = F_m + self.learning_rate * model.predict(X)
52
53         # calc loss
54         current_loss = self.binary_cross_entropy(y, self._sigmoid(F_m))
55
56         # save the model and loss
57         self.models.append(model)
58         self.loss.append(current_loss)
59
60     return self

```



3. a) $h(\bar{x}; \bar{\theta}^{(1)}) = \text{sign}(-1(x_2 - 0.17))$ with $\bar{\theta}^{(1)} = [2, 0.17, -1]^T$

i	$x_i^{(1)}$	$y_i^{(1)}$	
1	0.15	+1	✓
2	0.15	+1	✓
3	0.17	+1	X
4	0.17	-1	X
5	0.18	+1	X
6	0.19	-1	✓
7	0.20	-1	✓

$0.17 = x_2(F)$
 $0.17 > x_2(-1)$
 $0.17 < x_2(+1)$

$$\epsilon_1 = \sum_{i: y_i \neq h(x_i)} w_i^{(0)} = \frac{1}{7} + \frac{1}{7} + \frac{1}{7} = \frac{3}{7}$$

$$a_1 = \frac{1}{2} \ln \left(\frac{1 - 3/7}{3/7} \right) = 0.14384$$

$$w_i' = w_i^{(0)} \exp(-a_1 \cdot y_i \cdot h(x_i)) = \begin{cases} \frac{1}{7} e^{-a_1} & \checkmark \\ \frac{1}{7} e^{a_1} & \times \end{cases}$$

$$w_i^{(1)} = \frac{w_i'}{\sum_j w_j'} = \begin{cases} \frac{1}{8} & \checkmark \\ \frac{1}{6} & \times \end{cases}$$

SO, $(\bar{w}_1^{(1)}, \dots, \bar{w}_7^{(1)}) = (\frac{1}{8}, \frac{1}{8}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{8}, \frac{1}{8})$

b) $h_1(x) = \text{sign}(0.205 - x_0)$, $a_1 = 0.846$

$h_2(x) = \text{sign}(0.185 - x_1)$, $a_2 = 1.199$

$h_3(x) = \text{sign}(0.155 - x_1)$, $a_3 = 1.557$

loss: 1.557

$$c) \bar{w}_i^{(m)} = \frac{w_i^{(m)} \exp(-a_m y_i h_m(x_i))}{\sum_j w_j^{(m)} \exp(-a_m y_j h_m(x_j))} = \frac{\epsilon_m e^{a_m}}{(1 - \epsilon_m) e^{-a_m} + \epsilon_m e^{a_m}}$$

using $e^{a_m} = \sqrt{\frac{1 - \epsilon_m}{\epsilon_m}}$ and $e^{-a_m} = \sqrt{\frac{\epsilon_m}{1 - \epsilon_m}}$

$$= \frac{\epsilon_m \sqrt{\frac{1 - \epsilon_m}{\epsilon_m}}}{(1 - \epsilon_m) \sqrt{\frac{\epsilon_m}{1 - \epsilon_m}} + \epsilon_m \sqrt{\frac{1 - \epsilon_m}{\epsilon_m}}}$$

$$= \frac{\sqrt{\epsilon_m(1 - \epsilon_m)}}{2\sqrt{\epsilon_m(1 - \epsilon_m)}}$$

$$= \frac{1}{2}$$

d) Because the error is $1/2$, the current weak classifier is no better than a random guess. This is not a worry though because the next round of updates will learn something new and potentially have a loss < 0.5 which would make it a better-than-random classifier.


```

56 def adaboost(X: np.ndarray, labels: np.ndarray, num_iterations: int) -> tuple[list[tuple[int, float, str]], np.ndarray]:
57     """Train an AdaBoost ensemble of decision stumps.
58
59     Args:
60         X: training data
61         labels: training labels
62         num_iterations: how many weak learners to train
63
64     Returns:
65         a (functions, alpha) tuple where functions is a list of decision stumps and alpha is the weights of
66         each stump
67     """
68     n, d = X.shape
69     w = np.ones(n) / n
70     alpha = np.zeros(num_iterations)
71     functions = []
72     for i in range(num_iterations):
73         feature, threshold, operator = best_decision_stump(w, X, labels)
74         print(f"decision stump {i}: x_{feature} {operator} {threshold}")
75
76         # TODO: implement the rest of the AdaBoost algorithm
77         preds = stump_classification_result(feature, threshold, operator, X)
78
79         err = float(w @ (preds != labels))
80
81         eps = 1e-12
82         err = np.clip(err, eps, 1 - eps)
83
84         a = np.log((1 - err) / err) / 2
85
86         w = w * np.exp(-a * labels * preds)
87
88         w = w / w.sum() # normalize
89
90         functions.append((feature, threshold, operator))
91
92         alpha[i] = a
93
94         print(f"iteration {i}: stump h(x)=sign({operator} {threshold} on x_{feature}), "
95               f"err={err:.4f}, alpha={a:.5f}")
96
97     return functions, alpha
98
99
100 def classify(functions: list[tuple[int, float, str]], alpha: np.ndarray, X: np.ndarray) -> np.ndarray:
101     """Make predictions from the ensemble of decision stumps.
102
103     Args:
104         functions: list of trained decision stumps
105         alpha: weights of each decision stump
106         X: input data
107
108     Returns:
109         predicted {-1, 1} labels
110     """
111     # TODO: implement classification using the trained ensemble
112
113     if not functions:
114         return np.ones(X.shape[0], dtype=int)
115
116     F = np.zeros(X.shape[0], dtype=float)
117     for a, (feature, threshold, compare) in zip(alpha, functions):
118         F += a * stump_classification_result(feature, threshold, compare, X)
119
120     y_hat = np.sign(F)
121     y_hat[y_hat == 0] = 1
122     return y_hat.astype(int)

```


4a) The scaled method yields much better results because it groups the points vertically $\begin{pmatrix} \circ \\ \circ \\ \circ \end{pmatrix}$ compared to the unscaled clustering model that groups horizontally $\begin{pmatrix} \circ & \circ \\ \circ & \circ \end{pmatrix}$. The standardized kmeans does better because it makes sure x_1 and x_2 contribute equally instead of letting x_2 be the main decider in the distance calculation.

b)

$$\begin{aligned} \text{i. } \bar{\mu}^{(0)} &= (0,0) \rightarrow \frac{(0,0) + (0,1) + (0,5)}{3} = \boxed{(0,2)} \\ \bar{\mu}^{(1)} &= (10,0) \rightarrow \boxed{(10,0)} \\ \bar{\mu}^{(2)} &= (10,1) \rightarrow \boxed{(10,1)} \end{aligned}$$

$$\begin{aligned} \text{ii. } \bar{\mu}^{(0)} &= (0,0) \rightarrow \frac{(0,0) + (0,1)}{2} = \boxed{(0,0.5)} \\ \bar{\mu}^{(1)} &= (10,0) \rightarrow \frac{(10,0) + (10,1)}{2} = \boxed{(10,0.5)} \\ \bar{\mu}^{(2)} &= (0,5) \rightarrow \boxed{(0,5)} \end{aligned}$$

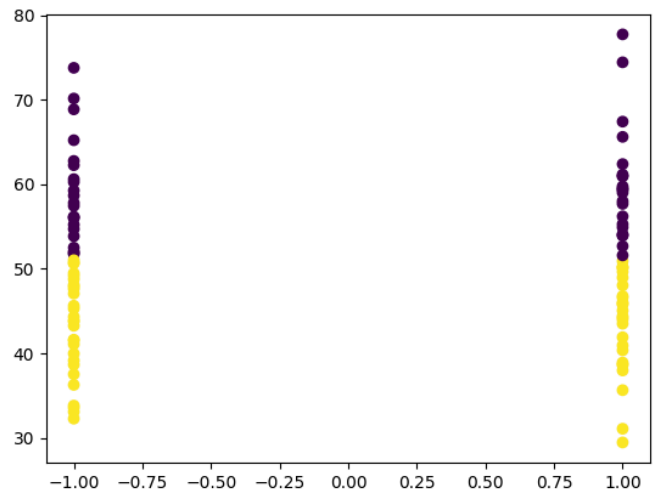
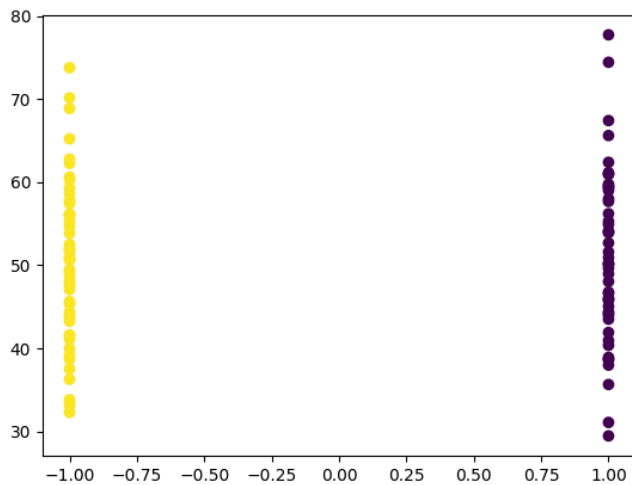
$$\begin{aligned} \text{iii. for (i) objective value} &= (2^2 + 1^2 + 3^2) + 0 + 0 = 14 \\ \text{for (ii) objective value} &= (0.5^2 + 0.5^2) + (0.5^2 + 0.5^2) + 0 = 1 \end{aligned}$$

So, the clustering in (ii) has a lower objective value.


```

7  if __name__ == "__main__":
8      # create the dataset
9      np.random.seed(445)
10     X = np.empty((100, 2))
11     X[:50, 0] = np.full(50, -1)
12     X[50:, 0] = np.full(50, 1)
13     X[:50, 1] = np.random.normal(loc=50, scale=10, size=50)
14     X[50:, 1] = np.random.normal(loc=50, scale=10, size=50)
15
16     # visualize KMeans clustering without standardization
17     kmeans = KMeans(n_clusters=2, init="k-means++", random_state=445)
18     klabs = kmeans.fit_predict(X)
19
20     plt.scatter(X[:, 0], X[:, 1], c=klabs)
21     plt.savefig("k-means.png")
22     plt.show()
23     plt.close()
24
25
26
27     # TODO: visualize KMeans clustering with standardization
28     scaler = StandardScaler()
29     X_scaled = scaler.fit_transform(X)
30     kmeans_scaled = KMeans(n_clusters=2, init="k-means++", random_state=445)
31     klabs_scaled = kmeans_scaled.fit_predict(X_scaled)
32
33     plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=klabs_scaled)
34     plt.savefig("k-means_scaled.png")
35     plt.show()
36     plt.close()

```



5.a)

i. Kmeans++ objective: 3551
Random objective: 12895

Kmeans++ gives a much smaller objective or inertia because it chooses initial cluster centers that are well spread out and converges to tight & well separated clusters.

ii. see plots on the next page...

iii. The Kmeans++ provides a much better cluster separation than random initialization. Building off what I said in (i), it looks like in our random initialization we assigned four cluster centers where it visually appears that we only have three clusters around $x = -60$ to $x = -40$. This caused us to separate the left most cluster into two. Also, we combined the north eastern most cluster with the cluster around $(0,0)$ into one because of the over separation on the left hand side.

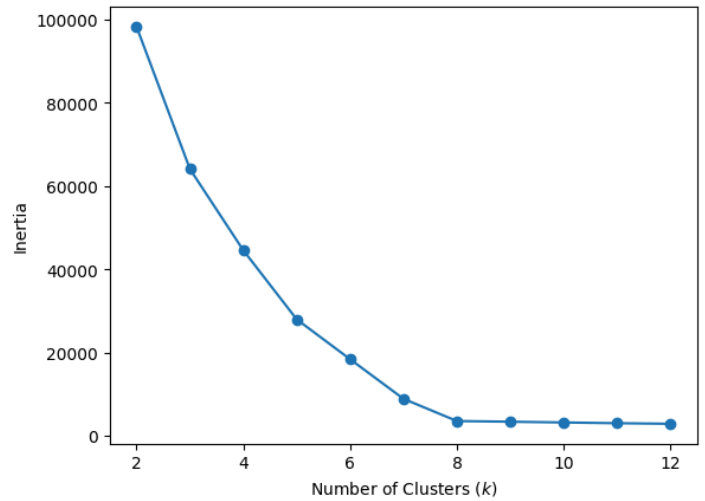
b) see the plot below...

The optimal number of clusters appears to be between 8 and 12, but in practice, taking the first value that doesn't lower the inertia is the best way to tune the number of clusters so that we don't overly separate the data. In this case, it would be $n=8$ which makes the most sense based on the other plots.

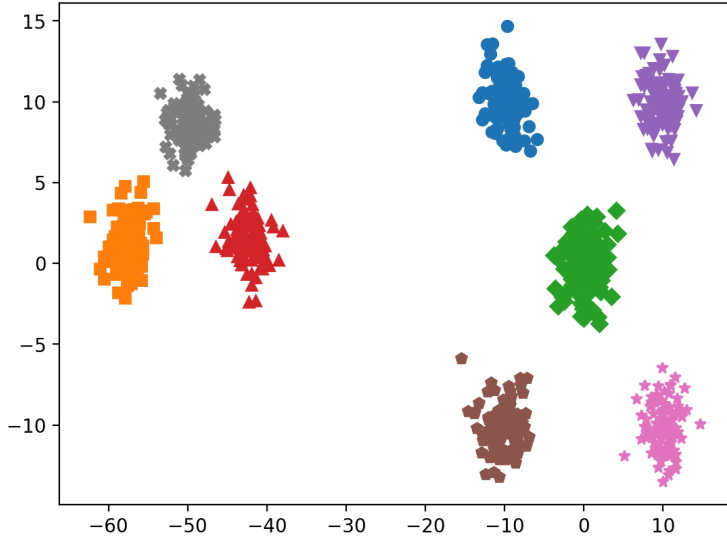

```

10 def visualize_kmeans(data: npt.NDArray, n_clusters: int, init: str) -> None:
11     """
12     Trains a k-means clustering model on the given data and visualizes the resulting clusters.
13
14     Args:
15     | data: A 2D NumPy array of shape (n_samples, n_features) representing the input data to be clustered.
16     | n_clusters: The number of clusters to form.
17     | init: The initialization method for the centers.
18
19     Returns:
20     | The trained scikit-learn KMeans object.
21     """
22
23     # TODO: Define a scikit-learn KMeans object
24     # - Set argument n_clusters (number of clusters) to n_clusters
25     # - Set argument init ('random' or 'k-means++') to init
26     # - Set random_state to 95
27     # - Set n_init to 10
28     kmeans = KMeans(n_clusters=n_clusters, init=init, random_state=95, n_init=10)
29
30     # Fit data to obtain clusters
31     kmeans.fit(data)
32
33     # TODO: Print final value of objective function
34     print(f"[{init}] k={n_clusters} objective: {kmeans.inertia_:.6f}")
35
36
37     # Plot each cluster on the same axes
38     plt.figure()
39     for cluster in np.arange(n_clusters):
40         plt.plot(data[kmeans.labels_ == cluster, 0], data[kmeans.labels_ == cluster, 1], PLT_MARKERS[cluster])
41     plt.title(f"K-Means Clustering Visualization - {init}")
42     plt.savefig(f"kmeans_visualization_{init}.png", dpi=200, bbox_inches="tight")
43     print(f"Plot saved to kmeans_visualization_{init}.png")
44
45
46 def plot_inertia(X: npt.NDArray, init: str) -> None:
47     """
48     Plot inertias over a range of number of clusters.
49     """
50
51     n_clusters = np.arange(2, 13)
52
53     inertias = []
54     for k in n_clusters:
55         # Keep the random state at 95 for clearer results
56         clf = KMeans(k, init=init, random_state=95).fit(X)
57         # TODO: Add the inertia for each fit classifier to the list of inertias
58         inertias.append(clf.inertia_)
59
60     plt.figure()
61     plt.plot(n_clusters, inertias, linestyle="-")
62     plt.scatter(n_clusters, inertias, marker="o")
63     plt.xlabel("Number of Clusters ($k$)")
64     plt.ylabel("Inertia")
65     plt.savefig("inertias.png", bbox_inches="tight")
66     print(f"Plot saved to inertias.png")
67
68
69 if __name__ == "__main__":
70     X = pd.read_csv("q5_data/data.csv").to_numpy()
71
72     # TODO: Implement visualize_kmeans and run the script with the appropriate init value above
73     visualize_kmeans(X, n_clusters=8, init="k-means++")
74     visualize_kmeans(X, n_clusters=8, init="random")
75
76     # TODO: Implement plot_inertia to generate a plot of KMeans losses with different numbers of clusters
77     plot_inertia(X, "k-means++")
78

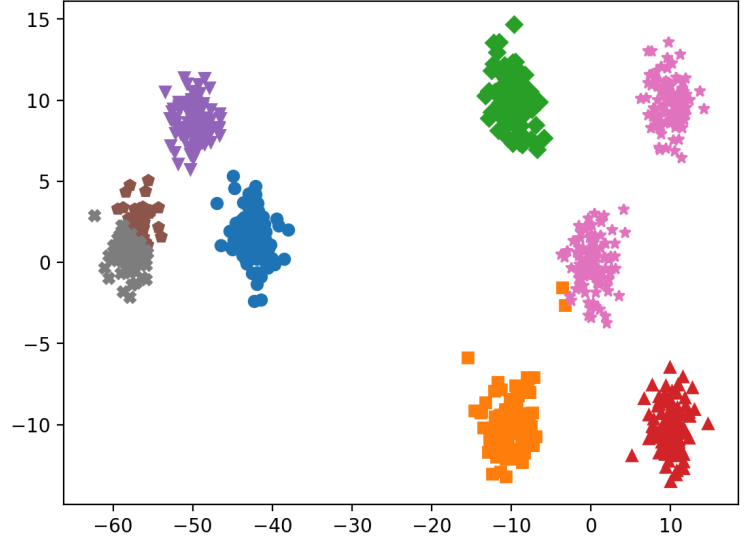
```



K-Means Clustering Visualization - k-means++



K-Means Clustering Visualization - random



$$b. a) P_K(x; \lambda) = \text{Pois}(x; \lambda) = \frac{\lambda^x e^{-\lambda}}{x!}$$

$$P_C(x; \mu) = \text{Pois}(x; \mu) = \frac{\mu^x e^{-\mu}}{x!}$$

$$\text{and } \Pr(\text{store}^{(i)} = K) = \gamma$$

$$\text{and } \Pr(\text{store}^{(i)} = C) = 1 - \gamma$$

$$L(\lambda, \mu, \gamma) = \prod_{i=1}^n \Pr(\text{store}^{(i)}, x^{(i)}) = \prod_{i \in K} \Pr(\text{store}^{(i)} = K) P_K(x^{(i)}; \lambda) \prod_{i \in C} \Pr(\text{store}^{(i)} = C) P_C(x^{(i)}; \mu)$$

$$= \prod_{i \in K} \left(\gamma \left(\frac{\lambda^{x^{(i)}} e^{-\lambda}}{x^{(i)}!} \right) \right) \prod_{i \in C} \left((1-\gamma) \left(\frac{\mu^{x^{(i)}} e^{-\mu}}{x^{(i)}!} \right) \right)$$

$$b) \ell(\lambda, \mu, \gamma) = \log(L(\lambda, \mu, \gamma))$$

$$= \sum_{i \in K} (\ln(\gamma) + \ln(P_K(x^{(i)}; \lambda))) + \sum_{i \in C} (\ln(1-\gamma) + \ln(P_C(x^{(i)}; \mu)))$$

$$= \underbrace{\sum_{i \in K} \ln(\gamma)}_{n_K \ln(\gamma)} + \underbrace{\sum_{i \in K} x^{(i)} \ln(\lambda)}_{m_K \ln(\lambda)} - \underbrace{\sum_{i \in K} \lambda}_{n_K \lambda} - \underbrace{\sum_{i \in K} \ln(x^{(i)}!)}_{S_K}$$

$$\begin{aligned} \ln(P_K(x^{(i)}; \lambda)) &= x^{(i)} \ln \lambda - \lambda - \ln(x^{(i)}!) \\ \ln(P_C(x^{(i)}; \mu)) &= x^{(i)} \ln \mu - \mu - \ln(x^{(i)}!) \end{aligned}$$

$$+ \underbrace{\sum_{i \in C} \ln(1-\gamma)}_{n_C \ln(1-\gamma)} + \underbrace{\sum_{i \in C} x^{(i)} \ln(\mu)}_{m_C \ln(\mu)} - \underbrace{\sum_{i \in C} \mu}_{n_C \mu} - \underbrace{\sum_{i \in C} \ln(x^{(i)}!)}_{S_C}$$

$$= n_K \ln(\gamma) + m_K \ln(\lambda) - n_K \lambda - S_K + n_C \ln(1-\gamma) + m_C \ln(\mu) - n_C \mu - S_C$$

$$c) \frac{\partial \mathcal{L}}{\partial \lambda} = \frac{m_K}{\lambda} - n_K = 0 \rightarrow \lambda = \frac{m_K}{n_K}$$

$$\frac{\partial \mathcal{L}}{\partial \mu} = \frac{m_C}{\mu} - n_C = 0 \rightarrow \mu = \frac{m_C}{n_C}$$

$$\frac{\partial \mathcal{L}}{\partial \gamma} = \frac{n_K}{\gamma} - \frac{n_C}{1-\gamma} = 0 \rightarrow \gamma n_C = (1-\gamma) n_K \rightarrow 0 = \gamma n_C + \gamma n_K - n_K \rightarrow \gamma = \frac{n_K}{n_C + n_K}$$

$$n_K = |K| = 4 \quad n_C = |C| = 6 \quad m_K = \sum_{i \in K} x^{(i)} = 23 \quad m_C = \sum_{i \in C} x^{(i)} = 21$$

$$\begin{aligned} \hat{\lambda} &= \frac{23}{4} = 5.75 \\ \hat{\mu} &= \frac{21}{6} = 3.5 \\ \hat{\gamma} &= \frac{4}{10} = 0.4 \end{aligned}$$